

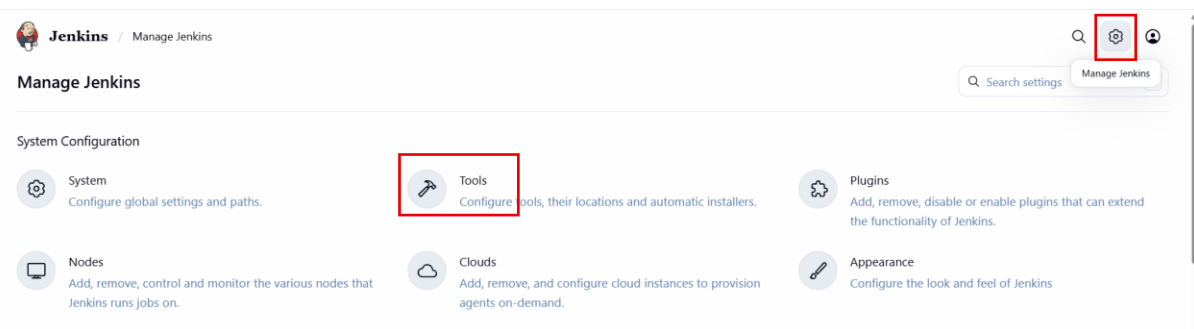
Freestyle Job Using Maven

To Begin with the Lab

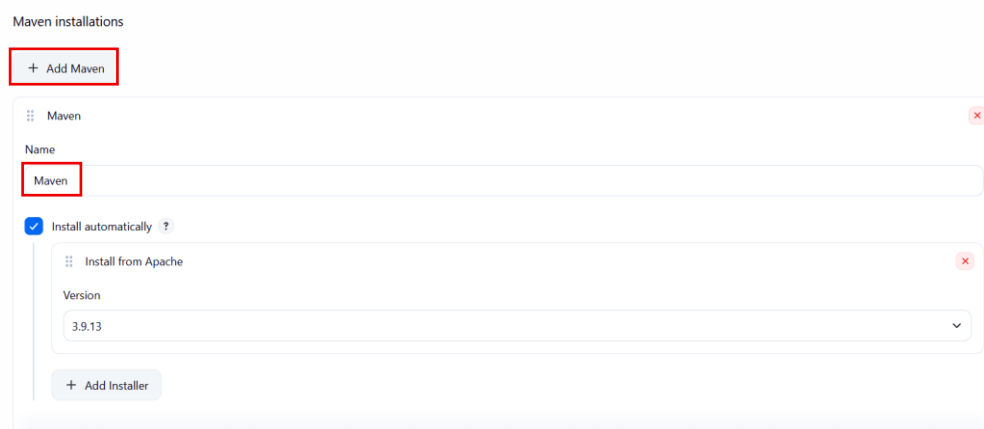
Summary of the Lab

This lab explains how to create a **Freestyle job in Jenkins using Maven**. First, open the Jenkins Dashboard, go to **Manage Jenkins** → **Tools**, and add a **Maven installation**, then save the configuration. Next, create a **New Item** and choose a **Freestyle project**. In **Source Code Management**, select **Git**, paste the repository URL from GitHub, and add credentials if the repository is private. Ensure the correct branch (main or master) is selected. In the **Build Steps**, add **Invoke top-level Maven targets** and set goals such as **compile**, **test**, and **package**. Save the job, run **Build Now**, and check **Console Output** and the **workspace/target folder** for build results.

- Repo Link [addressbook](#)
- Open **Jenkins Dashboard**.
- Click **Manage Jenkins**.
- Click **Tool**.



- Scroll to **Maven Installation**.
- Click **Add Maven**.



- Click **Save**.
- Go to **Jenkins Dashboard**.
- Click **New Item**.

New Item

Enter an item name

Maven-Freestyle

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

OK

- Navigate Source Code Management
- Select the git option from there

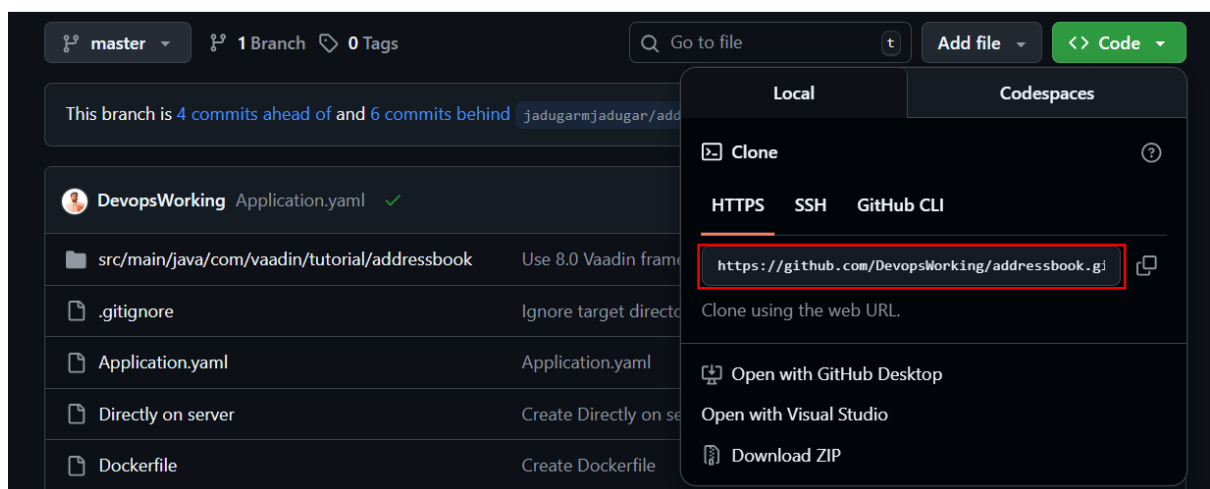
Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.

☐ None

☒ Git ?

- After selecting git.
- Copy the URL from the github



- Enter the Repository URL

- Note: If your repository is private, you have to select the credentials form the credential option.

Repositories ?

Repository URL ?

`https://github.com/DevopsWorking/addressbook.git`

❗ Please enter Git repository.

Credentials ?

- none -

+ Add

Advanced ▾

- Check you github repository is master or main.

Branches to build ?

Branch Specifier (blank for 'any') ?

`*/master`

- Navigate to the Build Step section
- Click on the Add Build Step
- Select the 'Invoke top-level Maven targets'

Environment

Build Steps

Post-build Actions

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

+ Add build step

Filter

Execute Windows batch command

Execute shell

Invoke Ant

Invoke Gradle script

Invoke top-level Maven targets

Run with timeout

Set build status to "pending" on GitHub commit

Invoke top-level Maven targets ?

Maven Version

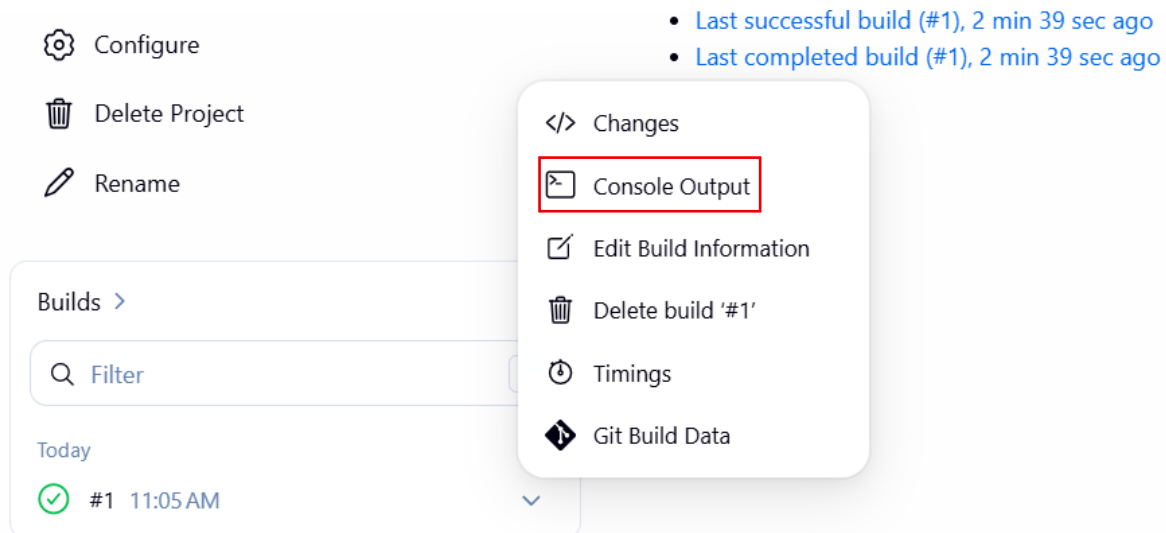
Maven

Goals

compile

Advanced ▾

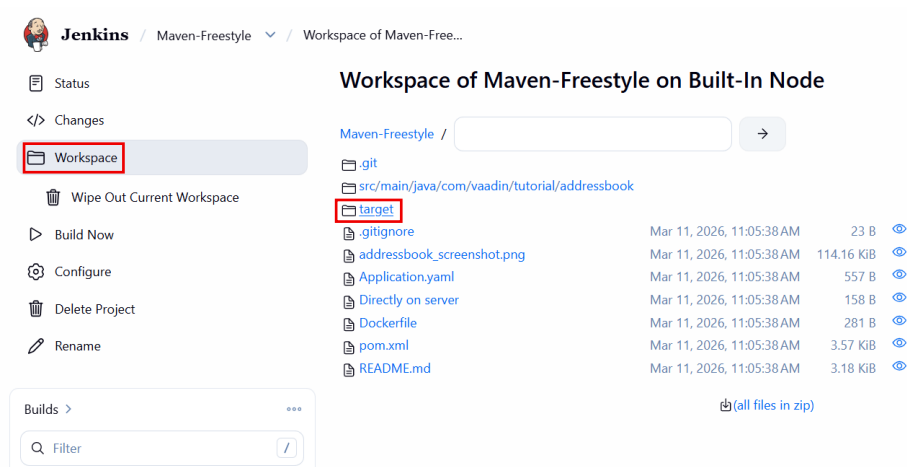
- Similarly make 3 Goal – compile, test and package with the Maven.
- Save the Job
- After saving it will create a job successfully
- Now click on the Build now
- Click on the Console Output to see the output



The screenshot shows the Jenkins console output menu. The menu is open, displaying several options. The 'Console Output' option is highlighted with a red box. The menu also includes options for 'Changes', 'Edit Build Information', 'Delete build '#1'', 'Timings', and 'Git Build Data'. In the background, the Jenkins console output is visible, showing a successful build with a green checkmark and the text '#1 11:05 AM'.

- [Last successful build \(#1\), 2 min 39 sec ago](#)
- [Last completed build \(#1\), 2 min 39 sec ago](#)

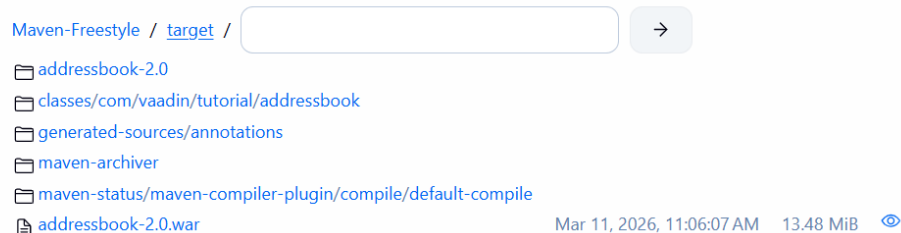
- Click on the workspace to check the packages.



The screenshot shows the Jenkins workspace view for the 'Maven-Freestyle' job. The 'Workspace' tab is selected and highlighted with a red box. The workspace contains several files and directories, including '.git', 'src/main/java/com/vaadin/tutorial/addressbook', 'target', '.gitignore', 'addressbook_screenshot.png', 'Application.yaml', 'Directly on server', 'Dockerfile', 'pom.xml', and 'README.md'. The 'target' directory is highlighted with a red box. The workspace is titled 'Workspace of Maven-Freestyle on Built-In Node'.

- Click on the target to check all the packages.

Workspace of Maven-Freestyle on Built-In Node



The screenshot shows the Jenkins target directory view for the 'Maven-Freestyle' job. The 'target' directory is selected, and the view displays a list of files and directories. The files and directories are listed with their names, sizes, and timestamps. The 'addressbook-2.0' directory is highlighted with a red box. The view is titled 'Maven-Freestyle / target /'.

- [addressbook-2.0](#)
- [classes/com/vaadin/tutorial/addressbook](#)
- [generated-sources/annotations](#)
- [maven-archiver](#)
- [maven-status/maven-compiler-plugin/compile/default-compile](#)
- [addressbook-2.0.war](#)